
Cloud Task Scheduling

Cameron Bailey and Bryant Peden

What is Cloud Task Scheduling?

- Efficiency-based process of managing tasks
 - Important due to the number of tasks systems and machines need to handle simultaneously
 - Balances time, energy consumption, and hardware capabilities
 - Helps systems and machines find the best performance
 - Creates better experience for users
 - Minimizes waste of time and resources
-
- Algorithms tested: Priority Scheduling, First In/First Out (FIFO), Round Robin, Grey Wolf Optimization (GWO)

Questions

1. How well does each algorithm scale, and how much optimization is appropriate?

- What is the growth rate of each algorithm?
- Is it worth it to optimize for each algorithm? Faster computing speeds require more overhead and more computational power, which can slow it down. What level of overhead is the fastest?

2. How steep is each algorithm's trade-off between energy consumption and performance?

- More available hardware leads to better, faster results. How will each algorithm perform when given the same level of hardware as the other algorithms? Which algorithms would require more hardware to perform well?

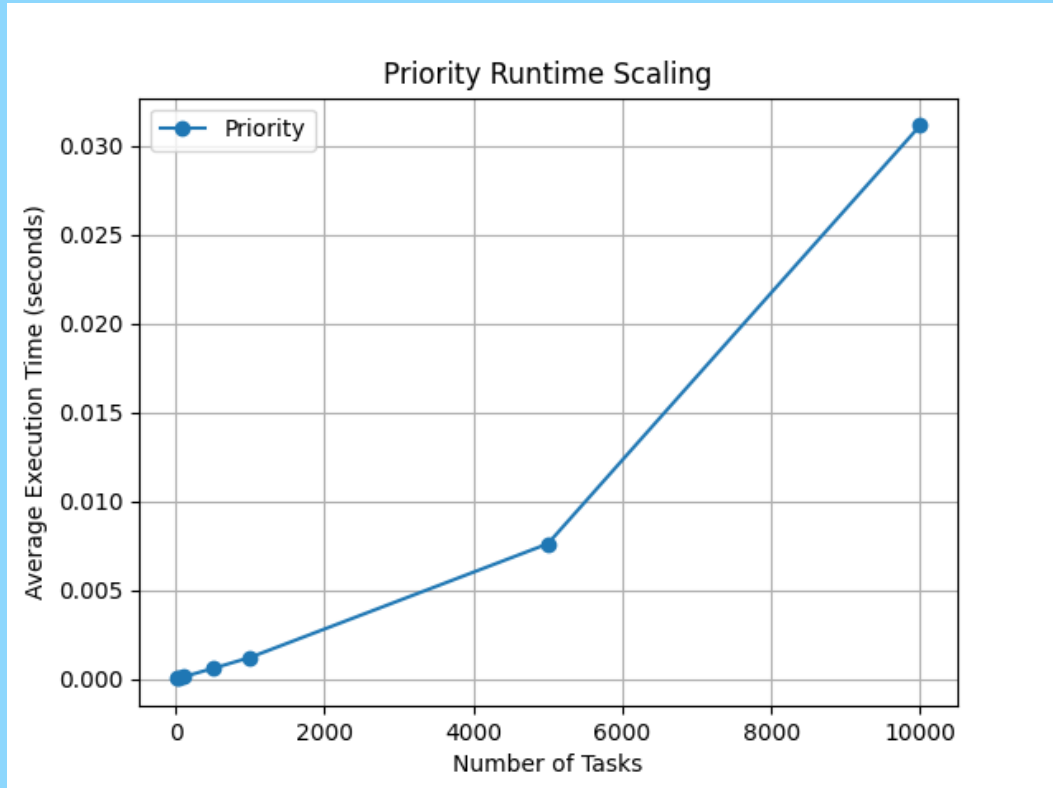
3. Which algorithms tend toward data locality as opposed to load balancing?

- Keeping your data in areas of the machine where that data already exists can make computation faster. If you spread the work among idle, unused parts of the machine, this can lighten the load, but it requires more overhead, which could potentially slow down computation.

Priority Scheduling

- Each task is given a priority level. The number of levels is decided on a per-case basis, with the highest priority usually being 1 (ex: 1 is high priority, 5 is low priority).
- Scheduler selects a task with the highest priority and runs that task until it is complete.
- Versions:
 - Preemptive: if a task is being run and a higher priority task is submitted, the task is interrupted.
 - Non-preemptive: if a task is being run, it will always finish, despite new submissions.
- Advantages
 - Reduces wait time for important tasks
 - Ensures that high-priority tasks are taken care of
- Disadvantages
 - Low-priority tasks are subject to "starvation", where they are ignored in favor of higher-priority tasks

Priority Scheduling

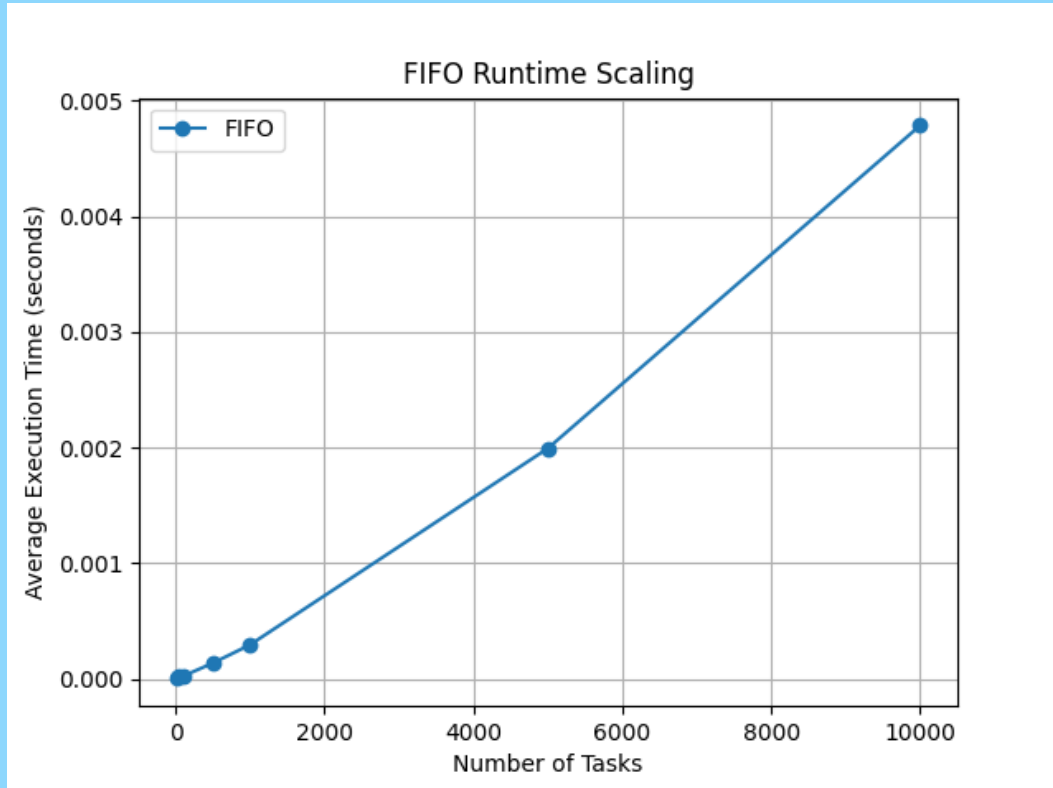


- Time Complexity:
 - $\Theta(n \log n)$ if implemented with priority queue
 - Basically simplified to $\Theta(n)$ at very small sizes
- Scaling is adequate, but is much improved when given proper overhead

First In/First Out

- System keeps track of tasks in a queue
- When space is needed for a new task, the task at the beginning of the queue is discarded
- Advantages
 - Extremely simple to implement and run
 - Cycles through tasks and pages very quickly, as no decisions need to be made
 - Consistent results
- Disadvantages
 - Commonly-used tasks and pages will be discarded despite their frequent use
 - "Convoy Effect": long tasks at the front of the queue prevent smaller tasks from being run

First In/First Out

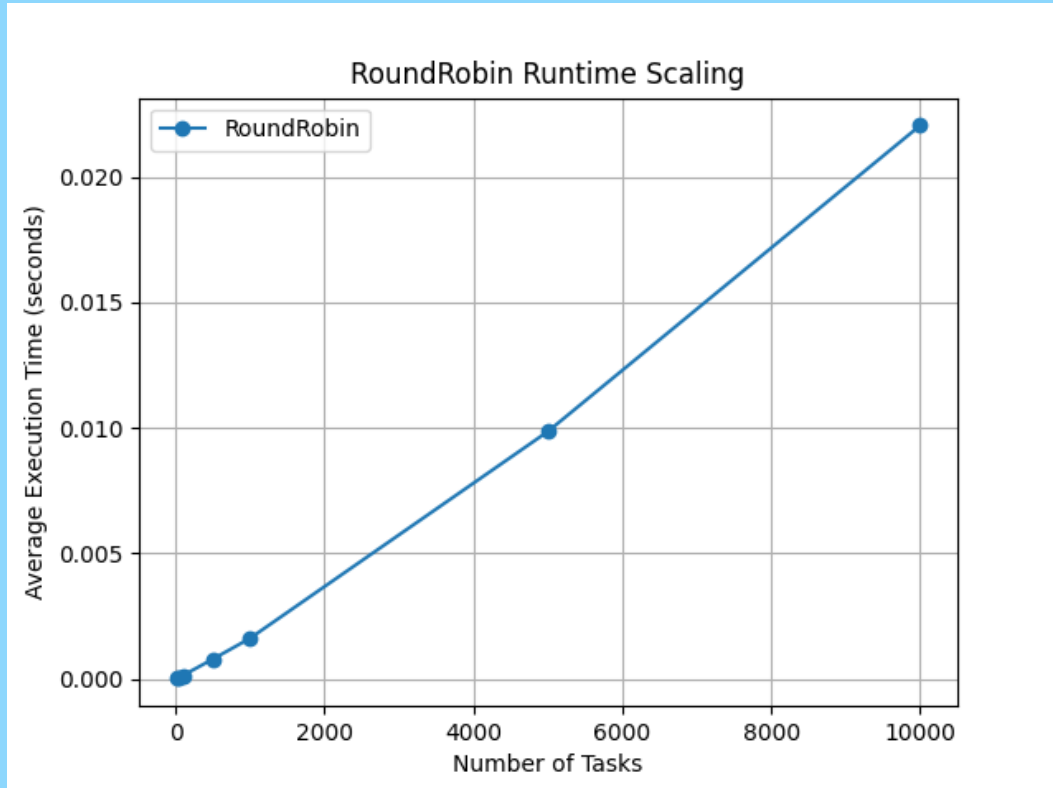


- Time Complexity: $\Theta(n)$
 - Handles tasks sequentially in the order it is given
- Doesn't adapt to repetition in larger input sizes

Round Robin

- System has a delta time, the amount of time each task is given before moving on to the next one
- Tasks are cycled through, spending the delta time on each one
- Cycle is repeated until each task is completed
- Advantages
 - Fair to all tasks, preventing any starvation
 - Friendly to users, as machines can have multiple tasks running at once
- Disadvantages
 - The more tasks there are, the longer it takes each individual task to complete, as it has to wait on the others
 - Considerable overhead required to manage time and constant switching
 - Important tasks are completed at the same rate as unimportant ones

Round Robin

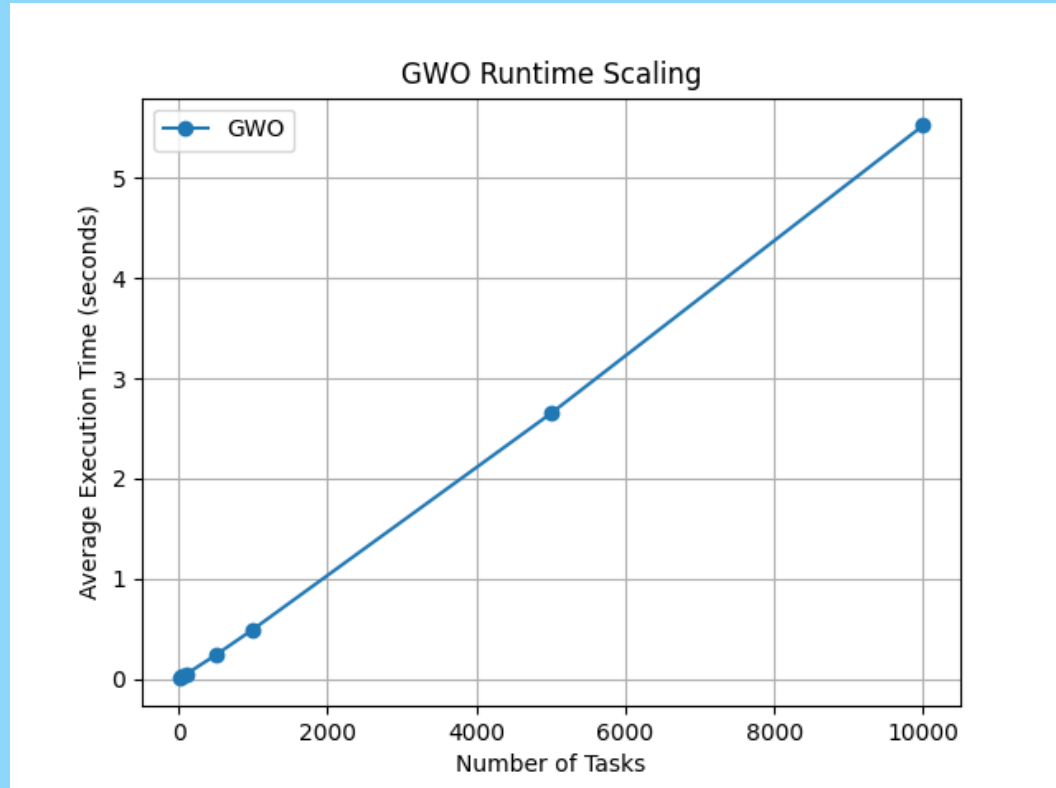


- Time Complexity:
 - Dependent on length of delta time
 - Low deltas require more switching, which can increase wait times
 - High deltas can frustrate users, as their tasks aren't running as efficiently from their point of view
 - Can be as low as $\Theta(n)$ in the best case
- Scaling greatly improved with proper overhead

Grey Wolf Optimization

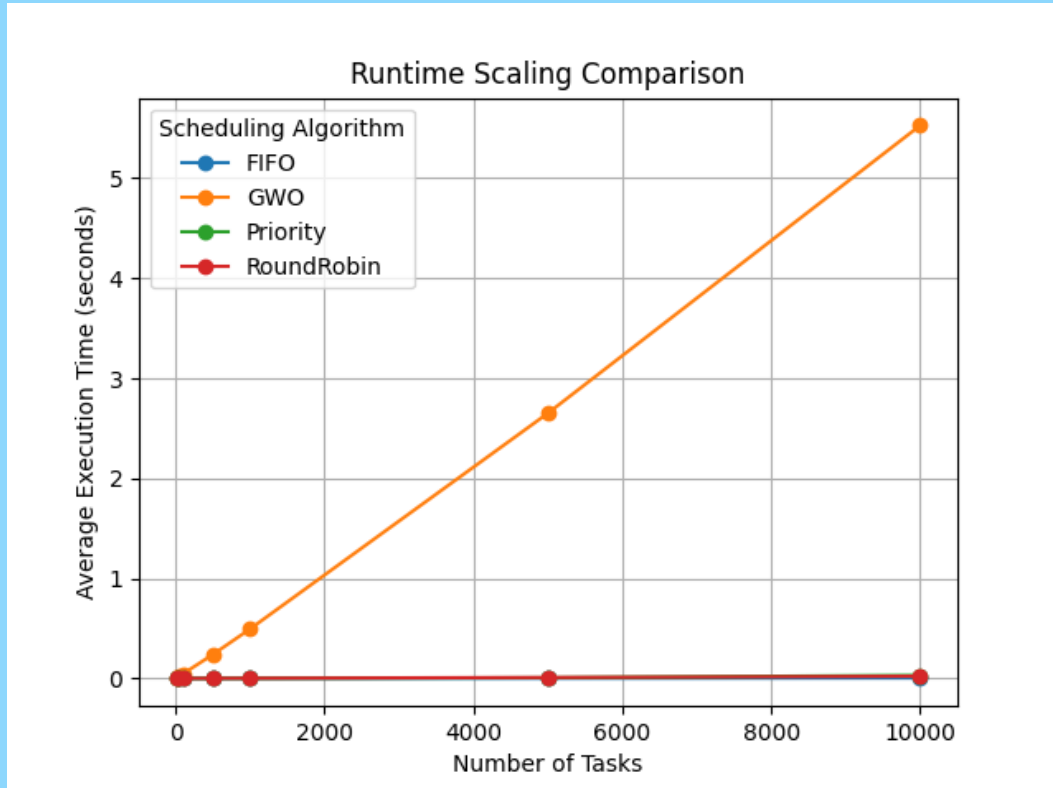
- Based on the way wolves organize themselves in the wild
- Searches for optimal task order solutions before implementation
- Attempts to minimize task waiting times, evaluates on a basis of total waiting time
- Assigns label based on how efficient the solution is
 - Alpha: best solution. Beta: second best. Delta: third best. Omega: all other solutions
- Advantages
 - Flexible, fit for complex scheduling problems
- Disadvantages
 - Significant overhead
 - Can be inconsistent, requiring fine-tuning for the situation

Grey Wolf Optimization



- Time Complexity: $\Theta(\text{iterations} * \# \text{ wolves})$
- Technically linear, but the coefficient is so large it effectively ruins the advantage at lower input sizes
- Can be extremely slow because of degree of overhead required

Comparing Algorithms



- For very small datasets, FIFO has the best time complexity.
- Round Robin was comparable to FIFO at the beginning, becoming slightly worse the more tasks it was given.
- Priority Scheduling scaled slightly worse than FIFO and RR due to its time complexity.
- At our input sizes, GWO was consistently worse than the other algorithms due to the extensive overhead. We may have seen better results with larger input sizes, but we wanted to use the same size for every algorithm.

Answering our Questions

1. How well does each algorithm scale, and how much optimization is appropriate?

- FIFO needs the least optimization to be efficient, followed by Priority Scheduling, Round Robin, and GWO.

2. How steep is each algorithm's trade-off between energy consumption and performance?

- The same order applies to energy efficiency, but when you give the less efficient algorithms sufficient energy, the performance improves drastically.

3. Which algorithms tend toward data locality as opposed to load balancing?

- FIFO and Priority Scheduling tend toward data locality. Round Robin leans heavily toward load balancing, while GWO can optimize for either.

Sources

- <https://www.geeksforgeeks.org/dsa/fifo-first-in-first-out-approach-in-programming/>
- <https://www.geeksforgeeks.org/operating-systems/difference-between-priority-scheduling-and-round-robin-rr-cpu-scheduling/>
- https://www.researchgate.net/publication/318185247_Grey_Wolf_Optimization_GWO_Algorithm
- <https://os.ecci.ucr.ac.cr/slides/Abraham-Silberschatz-Operating-System-Concepts-10th-2018.pdf>
- ChatGPT-5.5